

Programming Assignment II

CS683: Advanced Computer Architecture, Autumn 2025

Task 1: Offset-based Data Prefetching

Data Logs

Trace1:

Table size:32

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.529507	1	40.5813	10.008
Ip-Stride	0.545239	1.029	40.6049	6.39528
Offset-based	0.586429	1.108	40.6191	5.72336

Table size:64

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.529507	1	40.5813	10.008
Ip-Stride	0.545239	1.029	40.6049	6.39528
Offset-based	0.586429	1.108	40.6191	5.72336

Table size:128

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.529507	1	40.5813	10.008
Ip-Stride	0.545239	1.029	40.6049	6.39528
Offset-based	0.586398	1.106	40.6138	5.7466

Trace 2:

Table size:32

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.46276	1	47.742	11.8219
Ip-Stride	0.477382	1.031	47.7835	7.91908
Offset-based	0.505513	1.100	47.7634	7.039

Table size:64

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.46276	1	47.742	11.8219
Ip-Stride	0.477382	1.031	47.7835	7.91908
Offset-based	0.505513	1.100	47.7634	7.039

Table size:128

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.46276	1	47.742	11.8219
Ip-Stride	0.477382	1.031	47.7835	7.91908
Offset-based	0.505504	1.099	47.7611	7.0456

Trace 3:

Table size:32

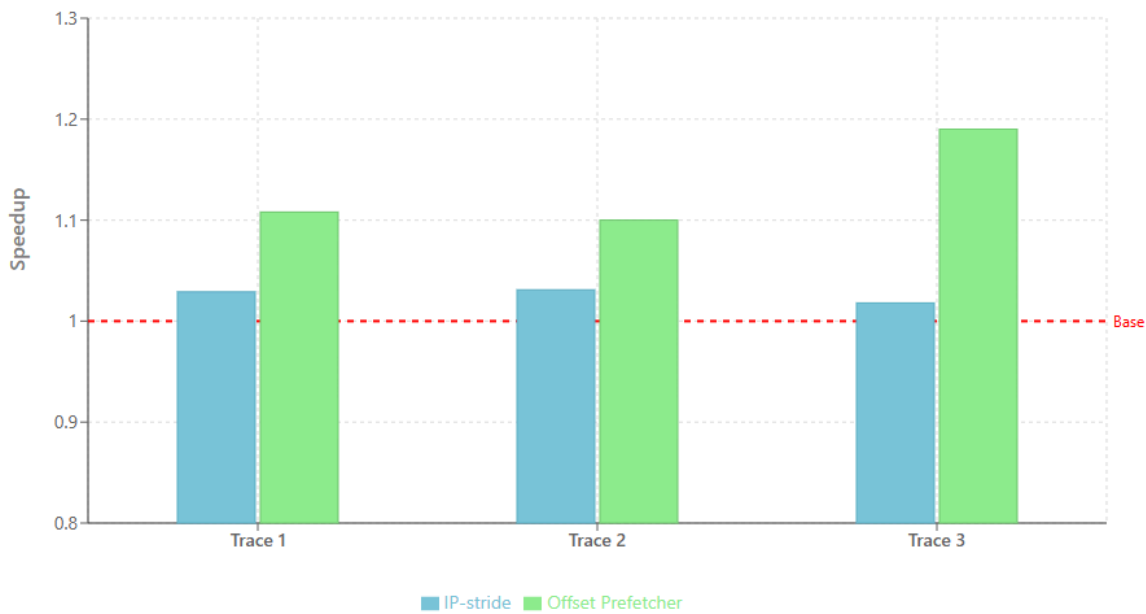
	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.839253	1	22.0985	11.8219
Ip-Stride	0.854738	1.018	22.0991	5.41756
Offset-based	0.991608	1.19	22.0979	2.3944

Table size:64

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.839253	1	22.0985	11.8219
Ip-Stride	0.854738	1.018	22.0991	5.41756
Offset-based	0.991608	1.19	22.0979	2.3944

Table size:128

	IPC	SpeedUp	L2 Total MPKI	L2 LOAD MPKI
Baseline	0.839253	1	22.0985	11.8219
Ip-Stride	0.854738	1.018	22.0991	5.41756
Offset-based	0.991608	1.19	22.0979	2.3944



Speedup of L2 Prefetchers over Baseline(no prefetching)

Key Observations and Insights:

- Trace 1: Offset-based prefetcher achieves **10.8% speedup** for all table sizes, which is much more than the 2.9% speedup achieved by the IP-stride prefetcher
- Trace 2: Offset-based prefetcher achieves **10% speedup** for all table sizes, as compared to the 3% speedup achieved by the IP-stride prefetcher.
- Trace 3: Offset-based prefetcher achieves **19% speedup** for all table sizes, as compared to the 1.8% speedup achieved by the IP-stride prefetcher.
- Non-inclusive cache hierarchy was used for all these simulations and observations.
- The dynamic learning mechanism of the offset-based prefetcher surely shows better results as compared to the fixed logic around the IP-stride prefetcher.
- The speedup achieved by any prefetcher also depends upon the load and the memory access pattern used in the evaluated code. Hence, we can only do a comparative study between different prefetchers using the same program, and there is no guaranteed value of speedup which can be targeted, as there is no direct relation between the performance of a program with the IP-stride prefetcher and the offset-based prefetcher.
- However, overall, the offset-based prefetcher outperforms the IP-stride prefetcher due to its ability to modify itself based on the current and past history of memory access patterns.

Logic:

The primary logic behind the offset-based prefetcher was that we have a table, in which each entry represents a page, and is indexed by the **Virtual Page Number** of the page, which will act as the signature. Each entry comprises of 5 offset values and the frequency of occurrence of each of the values. Each offset value also has an LRU bit associated with it. When an address in a page is accessed for the first time, an entry is allotted in this table. The next time when another address from the same page is requested, the offset is calculated from the previous address. This offset value is stored as one of the five values in the entry for this corresponding page. Similarly, five offset values are stored and their frequencies are incremented every time the same offset is observed. Each time, when prefetching, the offset value having the largest frequency, is selected and prefetched. In case, we get a different offset value apart from the five values stored, then one of the offset value is replaced using the LRU policy while also considering the frequency associated with that offset value.

Task 2: Exclusive Cache Hierarchy

Data Logs

Trace 1:

Table size:32

	IPC	SpeedUp	SpeedUp (Over non-inclusi ve)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.746038	1	1.408	10.0872	10.0085	10.008

Table size:64

	IPC	SpeedUp	SpeedUp (Over non-inclusi ve)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.746038	1	1.408	10.0872	10.0085	10.008

Table size:128

	IPC	Speedup	Speedup (Over non-inclusive)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.746038	1	1.408	10.0872	10.0085	10.008

Trace 2:

Table size:32

	IPC	Speedup	Speedup (Over non-inclusive)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.684685	1	1.47	11.9087	11.8224	11.8218

Table size:64

	IPC	Speedup	Speedup (Over non-inclusive)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.684685	1	1.47	11.9087	11.8224	11.8218

Table size:128

	IPC	Speedup	Speedup (Over non-inclusive)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.684685	1	1.47	11.9087	11.8224	11.8218

Trace 3:

Table size:32

	IPC	Speedup	Speedup (Over non-inclusive)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.94913	1	1.13	5.47968	5.41756	5.41756

Table size:64

	IPC	SpeedUp	SpeedUp (Over non-inclusive)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.949137	1	1.13	5.47968	5.41756	5.41756

Table size:128

	IPC	Speedup	Speedup (Over non-inclusi ve)	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.949137	1	1.13	5.47968	5.41756	5.41756

Key Observations and Insights

- The exclusive-cache hierarchy outperforms the non-inclusive cache hierarchy, as expected.
- This is because, on implementing exclusive cache, we increase the effective cache size as the same data is not present in multiple cache levels.
- Even without any kind of prefetching, the exclusive cache performs better as compared to the non-inclusive cache.
- Trace 1: The exclusive cache provides a massive **40% speedup** over the non-inclusive cache, without any prefetching.
- Trace 2: The exclusive cache provides an even larger **47% speedup** over the non-inclusive cache, without any prefetching.
- Trace 3: The exclusive cache provides a decent **13% speedup** over the non-inclusive cache, without any prefetching.
- This large difference between the speedups of Trace 1 and 2, and 3, could be because the workload defined by Trace 3 might not be that cache heavy, and hence, the implementation policy of the cache does not affect the speedup much.
- As observed in task 1, though, Trace 3 tends to have a workload that is affected more by prefetching as compared to the cache implementation. Hence, we achieve the maximum speedup by using the correct prefetcher.
- In this task, since we do not perform any prefetching, the speedup observed with Trace 3 is not much. However, improvement is still seen.
- All graphs are combined and presented with Task 3.

Logic

For implementing an exclusive cache, we perform modifications in the `handle_fill` function first. We add a block of code that firstly checks if the writeback queue of the lower-level cache is filled or not. In case it is filled, then it means that the lower level already has a lot of writebacks in its queue, and the currently evicted packet will not be accommodated. Otherwise, we make a writeback packet which has a fill level as one level lower than the current level, and with all other initialisations as required. The next change is done in the `handle_writeback` function. At the start of this function, the packet received is checked if it is already present in the L1 cache or not using another user-defined function called `check_exclusivity`. In case the function `check_exclusivity` returns false, then the packet is not considered in the writeback queue since it

is already there in another level. The next change is done in the `fill_cache` function, where again, in case the block is valid, then an eviction packet is created with proper initialisations, and the appropriate set in which the packet has to be sent to L1, i.e. higher level cache, upon which the packet is filled in that cache and removed from the lower level cache. The `check_exclusivity` function checks whether a packet is actually exclusive or not. It utilises the `get_way` function, and tries to get the set and way in which a packet is present. If a valid value is received, then it means that the block is already located in another cache, and hence it won't remain exclusive in case it is filled in the cache in which we are currently trying to fill.

Task 3: Data Prefetching for Exclusive

Cache Hierarchy

Data Logs

Trace 1:

Table size:32

	IPC	SpeedUp	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.746038	1	10.0872	10.0085	10.008
Ip-Stride	0.790046	1.058	9.13672	7.4278	6.7188
Offset-based	0.837996	1.23	10.0593	5.74176	3.18776

Table size:64

	IPC	SpeedUp	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.746038	1	10.0872	10.0085	10.008
Ip-Stride	0.790046	1.058	9.13672	7.4278	6.7188
Offset-based	0.838647	1.23	10.0593	5.74176	3.18776

Table size:128

	IPC	Speedup	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.746038	1	10.0872	10.0085	10.008
Ip-Stride	0.790046	1.058	9.13672	7.4278	6.7188
Offset-based	0.838647	1.23	10.0593	5.74176	3.18776

Trace 2:

Table size:32

	IPC	SpeedUp	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.684685	1	11.9087	11.8224	11.8218
Ip-Stride	0.727083	1.061	10.946	9.01336	8.06556
Offset-based	0.773444	1.129	11.8739	7.18368	3.78956

Table size:64

	IPC	SpeedUp	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.684685	1	11.9087	11.8224	11.8218
Ip-Stride	0.727083	1.061	10.946	9.01336	8.06556
Offset-based	0.773754	1.129	11.8739	7.18368	3.78956

Table size:128

	IPC	SpeedUp	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.684685	1	11.9087	11.8224	11.8218
Ip-Stride	0.727083	1.061	10.946	9.01336	8.06556
Offset-based	0.773754	1.129	11.8739	7.18368	3.78956

Trace 3:

Table size:32

	IPC	SpeedUp	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.94913	1	5.47968	5.41756	5.41756
Ip-Stride	1.01522	1.069	4.5788	3.47612	3.46552
Offset-based	1.07714	1.1348	5.46528	2.1262	1.64816

Table size:64

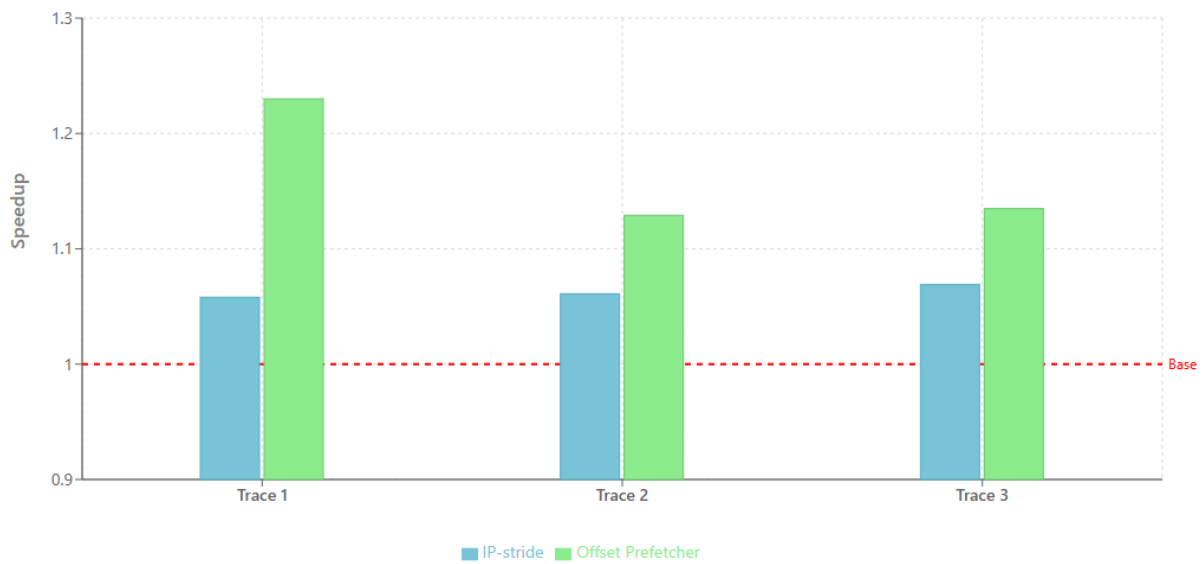
	IPC	SpeedUp	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.949137	1	5.47968	5.41756	5.41756
Ip-Stride	1.01522	1.069	4.5788	3.47612	3.46552
Offset-based	1.07726	1.1348	5.46528	2.1262	1.64816

Table size:128

	IPC	Speedup	L1D MPKI	L2 MPKI	LLC MPKI
Baseline	0.949137	1	5.47968	5.41756	5.41756
Ip-Stride	1.01522	1.069	4.5788	3.47612	3.46552
Offset-based	1.07726	1.1348	5.46528	2.1262	1.64816

Key Observations and Insights

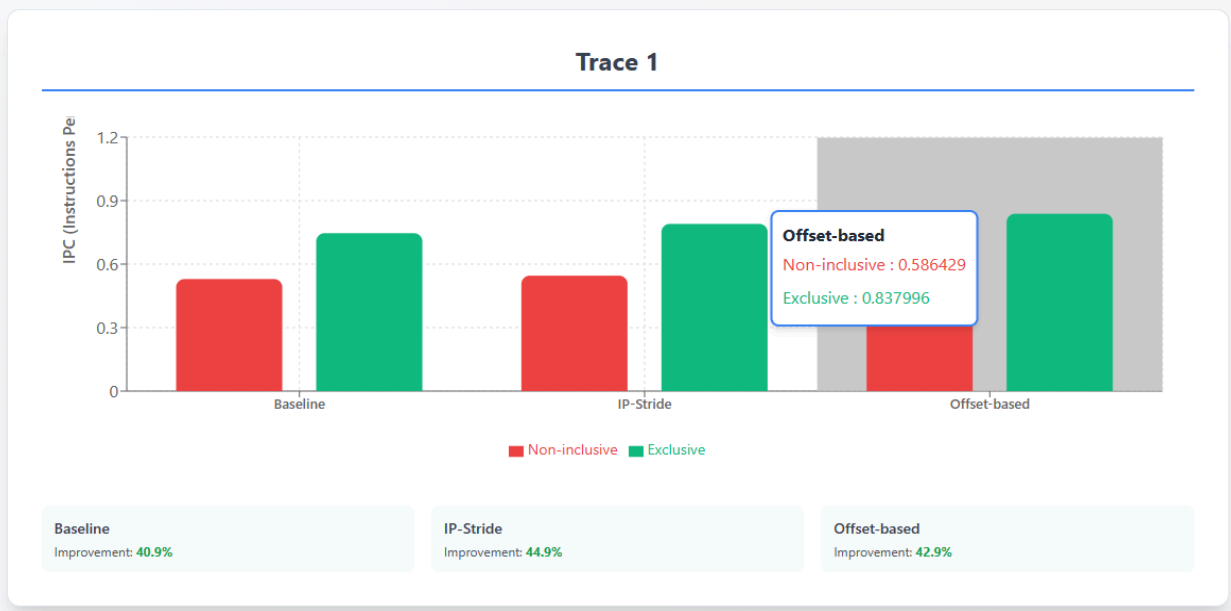
- When the offset-based prefetching and exclusive cache hierarchy are combined, we are expected to get the best results.
- Trace 1: The exclusive cache provides a massive **23% speedup** over the non-inclusive cache, with the offset prefetcher.
- Trace 2: The exclusive cache provides an even larger **12.9% speedup** over the non-inclusive cache, with the offset prefetcher.
- Trace 3: The exclusive cache provides a decent **13.5% speedup** over the non-inclusive cache, with the offset prefetcher.
- As we can see, the maximum amount of speedup is observed with trace 1, which shows that the workload for trace 1 responds the best to the prefetching and the exclusive cache hierarchy implementation.
- For trace 1 and 2, we can see improved speedup as compared to the case when we only had the prefetcher with non-inclusive cache.
- For trace 3, we see that the speedup has reduced as compared to the non-inclusive cache implementation. This is because, as we concluded in task 2, trace 3 depends a lot on the prefetching algorithm and not much on the cache implementation. However, using exclusive cache, is causing the prefetch algorithm to fetch unwanted data, which is not contributing much to the speedup.
- Exclusive cache consistently outperforms Non-inclusive across all traces and strategies
- Trace 3 shows the highest absolute IPC values, with Offset-based reaching **1.077 IPC**
- Performance gains from Exclusive over Non-inclusive range from **~40-50%** across configurations
- Offset-based prefetcher achieves the best IPC in both configurations across all traces



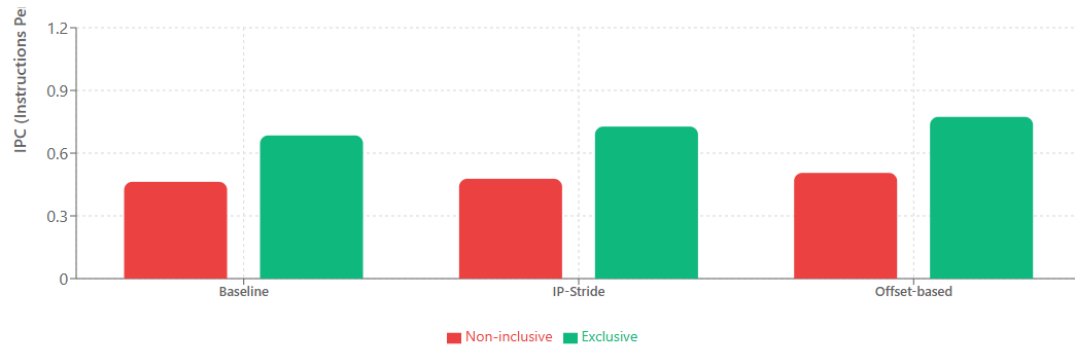
Speedup of L2 Prefetchers over Baseline (no prefetching)

IPC Comparison: Non-inclusive vs Exclusive Cache

Performance across different prefetcher strategies



Trace 2

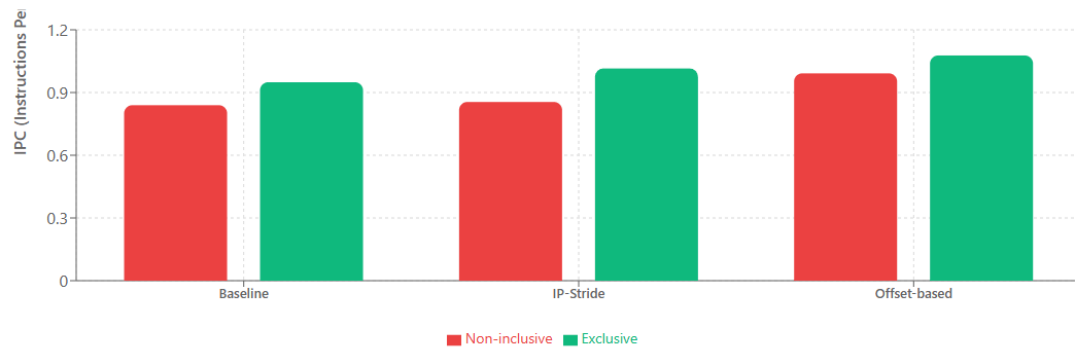


Baseline
Improvement: 48.0%

IP-Stride
Improvement: 52.3%

Offset-based
Improvement: 53.0%

Trace 3



Baseline
Improvement: 13.1%

IP-Stride
Improvement: 18.8%

Offset-based
Improvement: 8.6%